Bags, Vectors & Transformers

Scaling Up and Reproducibility · Bonus, Part 13

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

It Worked Last Month

While preparing this course in October 2026, three things broke in our own notebooks, without a single change to our code:

transformers 5

no longer accepted the `warmup_ratio` argument we used to fine-tune BERT

tokenizers

changed how overlapping chunks of long texts come back: silently the wrong windows

pandas 3

changed what `groupby(...).apply(...)` returns, which broke our data loading

Colab updates its pre-installed packages regularly. Unpinned code **will** break; the only question is when. And research has to give the same results a year later, on someone else's computer, when a reviewer asks.

Pin Everything: Packages, Models, Data

Packages

```
# requirements.txt
pandas==3.0.5
scikit-learn==1.9.1
transformers==5.19.0
```

Pin what you *import*, plus the Python version. `importlib.metadata` reads versions. `pip freeze` on Colab lists hundreds of irrelevant packages.

Models

```
from_pretrained(
"distilbert-base-uncased",
revision="<commit hash>")
```

Hub models and datasets are git repositories: the same name can change. For LLM APIs, record the model version string *and the date*.

Data

```
{"corpus.csv": {
 "sha256": "8165021808...",
 "bytes": 28}}
```

A checksum changes if one byte changes: proof that results come from *this* file. Raw data is **read-only**.

Beyond a requirements file: uv or conda lock files pin every dependency; Docker captures a whole system.

Randomness: Repeatable Is Not Robust

```
def set_seed(seed):
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
```

Splits, initialization, dropout, UMAP, gold-standard samples: randomness is everywhere. A seed makes a run **repeatable**.

GPUs aren't always deterministic, even with a seed: `torch.use_deterministic_algorithms(True)` helps, at a cost in speed.

But one seed is **one draw**. The same model on the same toy data, 10 random train/test splits:

mean accuracy	0.914
standard deviation	0.011
range	0.900–0.933

A 1-point difference between two models on one split could be noise. Compare on the same splits, over several seeds, and **report the spread**.

A Project Structure That Reruns

```
my_project/
  README.md   what, how, in which order
  requirements.txt
  config.json      paths, models, seeds
  data/
    raw/                    never edited
    processed/              made by code
    MANIFEST.json              checksums
  notebooks/                 exploration
  src/                 scripts, functions
  results/                 made by code
```

One rule above all: everything except raw data can be regenerated by code. No hand-edited spreadsheets in the middle of the pipeline.

- **relative paths**: `data/raw/...`, never `C:/Users/.../Desktop`
- settings in a **config file**: a new model or threshold is one edit, and the config documents the run
- notebooks to explore; scripts for the heavy, final steps

Caching, Chunks, and Jobs That Survive a Crash

Cache expensive steps

Embeddings, LLM calls, OCR: compute once, save to disk. Put a **hash of the settings** in the file name, so changing a setting recomputes instead of loading stale results.

```
embeddings_227f806cfa.npy  # seed=1
embeddings_4780f014c6.npy  # seed=2
```

Changed the *code*? Delete the cache, or add a version number to the settings.

Process in chunks, write each result at once



- constant memory, however big the corpus
- a crash costs one chunk: rerun **resumes**
- write to a temp file, rename when complete
- one bad document: log it and continue, never drop it silently
- chunks can run **in parallel** on a cluster

From Notebook to Script to Cluster

A **script** with command-line arguments and logging runs unattended, in order, with any settings:

```
python process_chunk.py \
-input data/raw/corpus.parquet \
-output results/parts \
-chunk 3 -n-chunks 10
```

Clusters: **Snellius** (SURF, the national supercomputer) for Dutch researchers; most universities also run their own (at WUR, ask about access to the HPC cluster, Anunna).

A **SLURM job array** runs the script once per chunk:

```
#!/bin/bash
#SBATCH -job-name=bvt-chunks
#SBATCH -time=01:00:00
#SBATCH -cpus-per-task=4
#SBATCH -mem=8G
#SBATCH -array=0-9        # 10 tasks; "0-24%5":  max 5 at
once
#SBATCH -output=logs/%x_%A_%a.out

module load Python/3.11    # names differ per cluster
source $HOME/venvs/bvt/bin/activate
python src/process_chunk.py ...  -chunk $SLURM_ARRAY_TASK_ID
-n-chunks 10
```

`sbatch` submits, `squeue -u $USER` shows, `scancel` cancels. Start from your cluster's own documentation.

Sharing: Version Control and Archives

git and GitHub

- every version of your code, with a message about what changed and why
- small, logical commits: "Fix leakage in the train/test split", not "update"
- `.gitignore`: keep data, models, caches and **API keys** out
- GitHub Desktop is the easiest start

Archive a snapshot with a DOI

- **Zenodo**: DOI for a GitHub release, direct integration
- **OSF**: pre-registrations, materials, data
- data repositories: 4TU.ResearchData, DANS

Text data often can't be shared in full: copyright, privacy, platform terms.

Share what you can:

- document IDs or URLs
- the code that collects and processes the data
- derived data: counts, topic proportions, labels
- a description of what's missing and why

Legal side: Bonus 3 (scraping) and 4 (anonymizing).

A Reproducibility Checklist

- ☐ README: what, how to run, in which order, how long, which hardware
- ☐ pinned `requirements.txt` and Python version
- ☐ models and datasets pinned (commit hashes; API model strings and dates)
- ☐ checksums of raw data; raw data never edited by hand
- ☐ seeds set; results over several seeds where randomness matters
- ☐ settings in a config file; no absolute paths
- ☐ expensive steps cached; long jobs chunked, resumable, logged
- ☐ code in git; a tagged release archived with a DOI
- ☐ data shared as far as legally possible, the rest documented

The real test: someone else, or you on a fresh Colab, reruns it from scratch. Everything else on this list is there to make that pass.

In the Notebook

- An environment report, a pinned `requirements.txt`, version checks
- Pinning models, and checksums for data files
- Seeds, and how much results vary between them
- A project skeleton with a config file
- Caching, chunked processing, crash recovery, error logs
- A command-line script and a SLURM job array

`bonus/notebooks/13_scaling_up_and_reproducibility_exercises.ipynb` · 5 exercises · no GPU needed